TRABAJO DE APLICACIÓN PROFESIONAL

Modelo de datos del sistema

Anexo 2 — Estructura de la base de datos NoSQL (Cloud Firestore)

Sistema de punto de venta, inventario y facturación electrónica
Anexo técnico · Documento generado a partir del código fuente

1. Introducción

1.1 Contexto del sistema

El sistema objeto de este trabajo es una aplicación de punto de venta, control de inventario y facturación electrónica que opera de forma continua en un establecimiento comercial. La interfaz de administración y venta es una *single page application* desarrollada en React con TypeScript, que no utiliza enrutador ni servidor de aplicación propio: toda la lógica de acceso a datos se ejecuta en el navegador y se comunica de manera directa con **Cloud Firestore**, el servicio de base de datos documental de Firebase. La autenticación se resuelve con Firebase Authentication y la emisión de comprobantes ante la Superintendencia Nacional de Aduanas y de Administración Tributaria (SUNAT) se delega en un servicio externo consumido por HTTP.

1.2 Elección de una base de datos NoSQL documental

La naturaleza operativa del negocio justifica el modelo documental por cuatro razones concretas, todas ellas verificables en la estructura de datos descrita en este anexo:

1. **El agregado natural es el documento, no la fila.** Una venta es una unidad indivisible de información: su cabecera, sus líneas de detalle, la forma de pago, el bloque de estado tributario y los datos del receptor se leen y se escriben siempre juntos. En el modelo implementado, todo ello vive en un único documento de la colección `tblDocumentos`, de modo que registrar una venta es una sola operación atómica y no una transacción sobre varias tablas relacionadas.
2. **El esquema de las líneas varía entre productos.** Una línea de venta puede corresponder a un producto con presentaciones y factor de conversión, a un componente de un *kit* o a un ítem manual de embalaje que no existe en el catálogo. La línea conserva los campos que le corresponden sin obligar a columnas nulas ni a tablas auxiliares.
3. **Sincronización en tiempo real y operación sin conexión.** El punto de venta necesita reflejar en pantalla los cambios de stock, los pedidos enviados a caja y el estado de los comprobantes en el momento en que ocurren. La biblioteca cliente de Firestore ofrece escuchas en vivo y una caché local persistente, que permite seguir operando de forma intermitente y sincronizar después.
4. **Ausencia de servidor propio.** Al no existir una capa de *backend* propia para el negocio, el modelo documental con reglas de seguridad declarativas evita administrar un motor relacional, sus migraciones y su dimensionamiento.

1.3 El prefijo `tbl` y la compatibilidad con la aplicación móvil

Los nombres de todas las colecciones del sistema comienzan con el prefijo `tbl`. El registro central de dichos nombres es el módulo `src/constants/colecciones.ts`, que constituye la fuente de verdad del sistema y declara dieciséis constantes de colección agrupadas en un único objeto congelado. La primera línea de ese archivo documenta explícitamente el motivo del prefijo: son las mismas colecciones que declara el módulo `constants/Colecciones.js` de la aplicación móvil.

El sistema no es una aplicación aislada. Existe una aplicación móvil desarrollada con Expo y React Native, y un servicio de facturación electrónica, que comparten la misma base de datos. El nombre de una colección es, en Firestore, la totalidad del contrato de acceso: no hay esquema central ni catálogo que valide una discrepancia. Mantener una denominación única y estable —con el prefijo `tbl` heredado de la versión móvil original— es la condición que permite que una venta registrada desde el teléfono sea leída, cobrada, reportada y enviada a SUNAT desde la aplicación web, y a la inversa. Renombrar una colección en una sola de las dos aplicaciones no produciría un error de compilación: produciría una base de datos aparentemente vacía en una de ellas. Por esa razón los

nombres se consumen siempre a través de las constantes de `colecciones.ts` y nunca como literales dispersos en el código, y por esa misma razón este anexo documenta los nombres exactos tal como están declarados.

> **Alcance del documento.** Todos los nombres de colección, de campo y los valores enumerados que aparecen en este anexo fueron verificados por lectura directa del código fuente del sistema. No se incluyen métricas de producción ni estimaciones: cuando un dato no es derivable del código, el anexo lo declara expresamente en lugar de suponerlo.

2. Inventario de colecciones

La Tabla 2.1 enumera las 16 colecciones declaradas en `src/constants/colecciones.ts`, su propósito funcional, el criterio de identificación del documento y el número de campos clave documentados. La sección 3 detalla por separado la colección `tblDocumentos`, que es el núcleo del sistema; por ese motivo su columna de recuento consigna únicamente los campos de cabecera.

Colección	Propósito	Documento que guarda	Campos
`tblProductos`	Catálogo de artículos con precios, existencias, presentaciones y datos tributarios.	Un documento por producto; identificador igual al código de barras o al código interno, o bien `P-<marca de tiempo>` cuando el código se deja vacío.	25
`tblMovimientos`	Kardex: todo movimiento de existencias (venta, compra, ajuste, anulación, nota de crédito).	Un documento por movimiento; identificador automático generado por Firestore.	20
`tblDocumentos`	Documentos de venta: notas de venta, boletas, facturas, notas de crédito y débito, y pedidos enviados a caja.	Un documento por comprobante o pedido; identificador compuesto por período y número (sección 3).	27*
`tblContadores`	Secuencias de numeración de comprobantes y de pedidos.	Un documento por espacio de numeración, con identificador normalizado (sección 5).	3
`tblTurnos`	Turnos de caja: apertura, acumulados, ventas y gastos del turno, cierre y arqueo.	Un documento por turno; identificador automático generado por Firestore.	26
`tblCierresCaja`	Registro histórico e inmutable del cierre de cada turno, con su análisis de diferencias.	Un documento por cierre; identificador automático, distinto del documento del turno.	22
`tblClientes`	Fichas de clientes, con datos fiscales, lista de precios, descuento y línea de crédito.	Un documento por cliente; identificador igual al número de documento (DNI o RUC).	24
`tblUsuarios`	Usuarios del sistema con rol, funciones habilitadas, caja asignada y series de comprobante.	Un documento por usuario; identificador igual al *uid* de Firebase Authentication.	17
`tblProveedores`	Fichas de proveedores, con datos de contacto y personas de referencia.	Un documento por proveedor; identificador igual al número de documento o al nombre normalizado.	11
`tblEgresos`	Egresos de caja y gastos del negocio, con su categoría y su vínculo con el turno.	Un documento por egreso; identificador automático generado por Firestore.	13
`tblHistorialPrecios`	Bitácora de cambios de precio de compra y de precio de venta.	Un documento por cambio registrado; se escribe solo	8

Colección	Propósito	Documento que guarda	Campos
		cuando el valor cambia efectivamente.	
`tblPromocionesVolumen`	Reglas de promoción automática por volumen sobre un conjunto de productos.	Un documento por promoción; identificador automático o el de la promoción editada.	10
`tblConfiguracion`	Parámetros del negocio, series de comprobante, datos tributarios y preferencias de operación.	Un único documento de configuración, `empresa`, dentro de la colección.	31
`tblCategorias`	Categorías de producto administradas, con su orden de presentación e icono.	Un documento por categoría; identificador igual al nombre normalizado a *slug*.	6
`tblErrores`	Registro de errores de la aplicación web, consultable desde el módulo de auditoría.	Un documento por error registrado; identificador automático generado por Firestore.	9
`tblCajas`	Puntos de emisión: cada caja declara las series con las que se numeran sus comprobantes.	Un documento por caja; identificador derivado del nombre (minúsculas, guiones bajos).	11

* Campos de cabecera del documento de venta. A ellos se suman los 32 campos del bloque `venta` y los 16 de la forma del ítem, detallados en la sección 3. Total documentado para esta colección: 75 campos.

2.1 Campos clave por colección

La Tabla 2.2 enumera los campos de cada colección. Los nombres se transcriben exactamente como se escriben en el código; cuando un campo solo se escribe en un flujo particular, se indica entre paréntesis.

Colección	Campos documentados	N.º
`tblProductos`	`nombre`, `codigo`, `categoria`, `precioVenta`, `precioCompra`, `precioEspecial`, `precioMinimo`, `preciosPorLista`, `descuentoPorcentaje`, `stock`, `minStock`, `unidadBase`, `tipoAfectacion`, `presentaciones`, `esFavorito`, `vecesVendido`, `activo`, `imagen`, `fechaVencimiento`, `isKit`, `kitItems`, `proveedorId`, `proveedorNombre`, `ultimaActualizacion`, `updatedAt`	25
`tblMovimientos`	`productoId`, `productoCodigo`, `productoNombre`, `categoria` (compras), `tipoMovimiento`, `cantidad`, `motivo`, `nota` (ajustes), `stockAnterior`, `stockNuevo`, `usuario`, `fecha`, `timestamp`, `presentacionUsada`, `presentacionId`, `cantidadPresentacion`, `factorConversion`, `metadata`, `documentoRef` (cobro de pedido), `serieNumero` (cobro de pedido)	20
`tblContadores`	`ultimoCorrelativo`, `serie`, `updatedAt`	3
`tblTurnos`	`usuarioId`, `usuarioNombre`, `cajaId`, `tipoTurno`, `fechaApertura`, `fechaCierre`, `estado`, `montoInicial`, `montoFinalEfectivo`, `efectivoEsperado`, `diferencia`, `ventasDelTurno`, `gastosDelTurno`, `totalVentas`, `totalEfectivo`, `totalTransferencia`, `totalGastos`, `totalGastosEfectivo`, `totalGastosTransferencia`, `vueltoTransferencia`, `totalAnulado`, `totalAnuladoEfectivo`, `totalAnuladoTransferencia`, `conciliacion`, `rectificacion`, `createdAt`	26
`tblCierresCaja`	`turnoId`, `usuarioId`, `usuarioNombre`, `tipoTurno`, `fechaApertura`, `fechaCierre`, `montoInicial`, `totalVentas`, `totalEfectivo`, `totalTransferencia`, `totalGastos`, `totalGastosEfectivo`, `totalGastosTransferencia`, `efectivoEsperado`, `montoFinalEfectivo`, `diferencia`, `analisis`, `observaciones`, `conciliacion`, `ventas`, `gastos`, `createdAt`	22
`tblClientes`	`tipoDoc`, `numeroDoc`, `razonSocial`, `razonSocialLower`, `direccion`, `email`, `telefono`, `fechaRegistro`, `ultimaCompra`, `totalCompras`, `montoTotal`, `frecuencia`, `tipoDocDefecto`, `listaPrecio`, `creditoActivo`, `limiteCredito`, `diasCredito`, `saldoPendiente`, `bloqueado`, `notas`, `sunatValido`, `sunatEstado`, `descuentoPorcentaje`, `updatedAt`	24
`tblUsuarios`	`email`, `nombre`, `rol`, `activo`, `permisos`, `cajaId`, `serieBoleta`, `serieFactura`, `seriePrefijo`, `serieBoletaNC`, `serieBoletaND`, `serieFacturaNC`, `serieFacturaND`, `activeSession`, `dashboardWidgets`, `createdAt`, `updatedAt`	17
`tblProveedores`	`nombre`, `documento`, `tipoDocumento`, `direccion`, `telefono`, `correo`, `contactos`, `notas`, `activo`, `fechaRegistro`, `updatedAt`	11
`tblEgresos`	`monto`, `descripcion`, `categoria`, `fecha`, `tipoPago`, `rucProveedor`, `proveedorNombre`, `proveedorId`,	13

Colección	Campos documentados	N.º
	`comprobante`, `usuarioId`, `usuarioNombre`, `turnoId`, `createdAt`	
`tblHistorialPrecios`	`productoId`, `productoNombre`, `campo`, `valorAnterior`, `valorNuevo`, `usuario`, `fecha`, `timestamp`	8
`tblPromocionesVolumen`	`nombre`, `tipo`, `valor`, `descuento`, `productoIds`, `activo`, `fechaInicio`, `fechaFin`, `notas`, `creadoEn`	10
`tblConfiguracion`	`nombre`, `ruc`, `direccion`, `correo`, `telefono`, `sucursal`, `mensaje`, `serie_prefijo`, `serieBoleta`, `serieFactura`, `logo_url`, `tipoRegimen`, `regionExonerada`, `conteoCiego`, `sunatApiUrlProd`, `sunatApiUrlTest`, `consultaApiBase`, `serieBoletaNC`, `serieBoletaND`, `serieFacturaNC`, `serieFacturaND`, `tipoAfectacionManual`, `puntoEmision`, `topeBoletaDni`, `descuentoMaximoPorcentaje`, `servidor`, `envioHabilitado`, `zonasEnvio`, `plantillasManuales`, `listasPrecios`, `updatedAt`	31
`tblCategorias`	`nombre`, `orden`, `activo`, `icono`, `createdAt`, `updatedAt`	6
`tblErrores`	`mensaje`, `contexto`, `fecha`, `usuario`, `usuarioId`, `ruta`, `dispositivo`, `stack`, `detalle`	9
`tblCajas`	`nombre`, `activo`, `serieBoleta`, `serieFactura`, `seriePrefijo`, `serieNC`, `serieND`, `serieNCB`, `serieNDB`, `createdAt`, `updatedAt`	11

2.2 Estructuras anidadas de uso frecuente

Cuatro campos de las colecciones anteriores almacenan, a su vez, estructuras con forma propia. Se describen por separado para no repetirlas en cada fila de las tablas previas.

Campo	Contenido
`pagos` (en `venta`)	Lista de pagos aplicados. Cada elemento tiene `tipo` (bolsillo: `'efectivo'`, `'transferencia'` o `'credito'`), `medio` (detalle: `'efectivo'`, `'yape'`, `'plin'`, `'transferencia'` o `'tarjeta'`), `monto` y `operacion` (número de operación, solo en medios electrónicos).
`presentaciones` (en producto)	Lista de presentaciones de venta o compra. Cada elemento tiene `id` (nombre normalizado), `nombre`, `factor` (unidades base por presentación), `esVenta` y `esCompra`.
`kitItems` (en producto y en la línea de venta)	Lista de componentes. Cada elemento tiene `productoId` y `cantidad`; el descuento de existencias se aplica sobre los componentes, no sobre el *kit*.
`activeSession` (en usuario)	Objeto con `deviceId` y `at`. El sistema lo escribe en cada inicio de sesión y lo vigila: si el identificador de dispositivo difiere del actual, la sesión anterior se cierra.

2.3 Colecciones declaradas fuera del registro central

Además de las 16 colecciones de `src/constants/colecciones.ts`, el código declara dos colecciones cuyo nombre no vive en ese registro sino en el módulo que las usa. Se consignan aquí por completitud, dado que forman parte de la base de datos:

- `tblAuditLog` — declarada en `src/lib/audit.ts`. Registra la actividad relevante del sistema con los campos `usuario`, `usuarioId`, `accion`, `entidad`, `entidadId`, `detalle` y `timestamp`. Es el destino de los registros de auditoría invocados desde los servicios de venta, turnos, egresos, compras, inventario y cajas.
- `tblDisplaySessions` — declarada en `src/lib/customer-display.ts`. Sostiene la sincronización entre el punto de venta y la pantalla orientada al cliente, que se abre en otro dispositivo mediante un parámetro de consulta.

Ambas se rigen por los mismos criterios de acceso que el resto del modelo; el prefijo `tbl` se conserva también en ellas.

> **Campos declarados pero no escritos por la interfaz web.** El tipo `Producto` declara `descuentoPorcentaje`: ninguna pantalla de la aplicación web lo escribe, pero sí lo consume el motor de descuentos a partir del producto del catálogo. El tipo `Proveedor` declara `saldoPendiente`, `creditoActivo`, `limiteCredito` y `diasCredito`: las vistas que los leen caen a cero cuando el campo falta y ninguna pantalla del web los persiste. Se documentan como parte del modelo declarado, no como datos escritos por esta aplicación.

3. El documento de venta (`tblDocumentos`) en detalle

`tblDocumentos` es el núcleo del modelo. En ella no conviven únicamente los comprobantes de venta: la misma colección aloja las notas de venta, las boletas, las facturas, las notas de crédito y débito, las boletas consolidadas y los **pedidos** que un vendedor envía a caja antes de que exista un comprobante. Lo que distingue a cada clase de documento es el valor del campo `tipo` y `tipoDoc`, junto con la presencia o ausencia del bloque `sunat`.

3.1 Identificador del documento

El identificador se compone de forma determinista, con el período y el número, para que un comprobante sea localizable sin una consulta previa:

- Comprobante emitido en el punto de venta: `<año>-<mes>_<tipo>-<serie>-<correlativo>`, por ejemplo `2026-9_BOLETA-B001-000123`. El mes se compone sin relleno de ceros.
- Pedido enviado a caja: `<año>-<mes>_PED-000123`.
- Nota de crédito o débito: `<año>-<mes>_NC-<serie>-<correlativo>`.
- Boleta consolidada: `<año>-<mes>_BOLETA-<serie>-<correlativo>`.

3.2 Campos de cabecera

Campo	Tipo	Contenido y origen
`tipo`	texto	Clase de documento: `'NV'`, `'BOLETA'`, `'FACTURA'`, `'NC'`, `'ND'`, o `'PENDIENTE'` mientras es un pedido sin comprobante.
`tipoDoc`	texto	Espejo del anterior. Se conserva duplicado porque distintos módulos —reportes, consolidación, envío a SUNAT— leen uno u otro según su origen.
`serie`	texto	Serie del comprobante, resuelta desde la caja del usuario que emite o, en su defecto, desde la configuración de la empresa. Cadena vacía en un pedido.
`numero`	número	Correlativo numérico dentro de la serie. Cero en un pedido.
`correlativo`	número	Mismo valor que `numero` al emitir una venta. Las notas de crédito y débito lo escriben como cadena con relleno de ceros; todos los lectores resuelven el valor con `numero ?? correlativo`, por lo que la heterogeneidad no afecta las búsquedas.
`serieNumero`	texto	Número completo legible, por ejemplo `B001-000123`. En un pedido contiene el código `PED-000123`.
`fecha`	texto ISO	Fecha y hora de emisión, en formato ISO 8601 con zona horaria.
`anio`	número	Año de emisión, como entero.
`mes`	número	Mes de emisión, numerado de 1 a 12.
`anioMes`	texto	Período de emisión en formato `año-mes`. Es el campo de filtrado mensual de los reportes y, por su condición de igualdad, forma parte de varios índices compuestos.
`createdAt`	texto ISO	Marca de creación del documento, usada como criterio de orden cuando `fecha` no está disponible.
`estado`	texto	Situación del documento: `'pendiente'`, `'pagado'`, `'cancelado'` o `'anulada'`. Los comprobantes emitidos directamente en el punto de venta no escriben este campo. La sección 4 desarrolla el ciclo de vida.
`usuarioId`	texto	Identificador del usuario responsable del documento. Los reportes filtran por este campo cuando el usuario no tiene habilitada la vista global; al cobrar un pedido se reescribe con el cajero que lo cobró.
`vendedorId`	texto	Identificador de quien armó la venta. En una venta directa coincide con `usuarioId`.
`vendedorNombre`	texto	Nombre de quien armó la venta. En una venta directa, el mismo usuario que cobró.
`cajero`	texto	Nombre de quien cobró. Cadena vacía mientras el documento es un pedido.
`cajeroId`	texto	Identificador del cajero que cobró el pedido; se escribe en la transición a `'pagado'`.
`cajaId`	texto	Caja asociada al documento. En un pedido es la caja de destino (ruteo); al cobrarlo se reescribe con la caja que efectivamente emitió el comprobante.

Campo	Tipo	Contenido y origen
`turnoId`	texto	Turno de caja en el que se registró la operación. Cadena vacía en un pedido, que aún no afecta a la caja.
`fechaCobro`	texto ISO	Momento en que un pedido se convirtió en comprobante.
`motivoCancelacion`	texto	Motivo obligatorio con el que el cajero descartó un pedido.
`pedido`	texto	Código `PED-…` con el que el pedido llegó a caja. Se conserva tras el cobro para poder rastrear el pedido original una vez que `serieNumero` pasa a contener el comprobante real.
`anulacion`	objeto	Bloque de trazabilidad de la anulación o de la cancelación. Contiene `fecha`, `usuario` y `motivo`; al anular un comprobante agrega `documentoAnuladoRef`, y al cancelar un pedido agrega `comprobante: null` y `tocoStock: false`, que declaran de forma explícita que no hubo comprobante que anular ni existencias que reponer.
`sunat`	objeto	Estado tributario del comprobante. Contiene `serie`, `numero`, `puntoEmision` y `estado` (`'pendiente'`, `'enviando'`, `'aceptado'`, `'rechazado'` o `'error'`); tras el envío se agregan `ultimoEnvio`, `cdrCode`, `cdrDesc`, `hash` y `cdr`. No existe en las notas de venta ni en los pedidos, porque ninguno de los dos se declara ante la administración tributaria.
`consolidado`	booleano	Presente y verdadero en la boleta que agrupa las notas de venta del día.
`nvsConsolidadas`	lista	Identificadores de las notas de venta agrupadas en una boleta consolidada. Cada nota afectada recibe, además, en su propio bloque `sunat`, las marcas `consolidado`, `consolidadoAt` y `documentoConsolidadoId`.
`updatedAt`	texto ISO	Marca de la última modificación. Se escribe en las transiciones de estado, en la transferencia de pedidos entre cajas y en la corrección de créditos.

3.3 El bloque `venta`

El bloque `venta` concentra el contenido comercial del documento: qué se vendió, cuánto se cobró, cómo se pagó y a quién. Su forma es deliberadamente idéntica entre una venta directa, un pedido y una boleta consolidada, porque el comprobante impreso y el archivo enviado a la administración tributaria se construyen a partir de él sin distinciones.

Campo	Tipo	Contenido
`id`	texto	Identificador de la venta dentro del turno. Es el valor con el que la anulación localiza la fila correspondiente en `ventasDelTurno`. Una venta directa lo deriva de la marca de tiempo; una boleta consolidada lo prefija con `consolidado_`.
`fecha`	texto ISO	Fecha y hora de la venta.
`items`	lista	Líneas de detalle. Su forma se describe en el apartado 3.4.
`total`	número	Importe total cobrado, **ya neto de descuentos**. Es el valor que el cobro, el comprobante y el arqueo consideran definitivo.
`importe_total`	número	Mismo importe que `total`, con el nombre heredado de la aplicación móvil. Se conserva por compatibilidad de lectura.
`descuentoTotal`	número	Descuento total aplicado. Es el nombre que leen el comprobante impreso y el generador del archivo tributario; su lectura está centralizada en una única función que, si el campo falta, recurre al nombre antiguo en notación con guion bajo.
`descuento_total`	número	Nombre antiguo del descuento total. Un pedido escribe **ambos** nombres para que el comprobante generado al cobrarlo declare el descuento que el vendedor había aplicado.
`descuentoGlobalTipo`	texto	Tipo del descuento manual cargado en el punto de venta: `'porcentaje'` o `'monto'`. Nulo cuando no se aplicó ninguno.
`descuentoGlobalValor`	número	Valor del descuento manual. Nulo cuando no se aplicó ninguno.
`descuento_global_tipo`	texto	Nombre con guion bajo del tipo de descuento global, escrito únicamente por el flujo de pedidos.
`descuento_global_valor`	número	Nombre con guion bajo del valor del descuento global, escrito únicamente por el flujo de pedidos.
`tipoPago`	texto	Forma de pago resultante: `'efectivo'`, `'transferencia'`, `'credito'` o `'mixto'` cuando hay más de un pago. En un pedido se escribe como cadena vacía, porque la forma de pago la decide el cajero al cobrar.
`forma_pago`	texto	Rótulo legible derivado de los pagos aplicados, por ejemplo el nombre del medio utilizado o el texto «Crédito». Con pago múltiple se compone a partir de los medios reales, para no rotular como transferencia una venta cobrada con dos medios distintos.
`pagos`	lista	Desglose de los pagos aplicados; la estructura de cada elemento se detalla en el apartado 2.2. Una venta a crédito nace con la lista vacía, porque todavía no se cobró nada.
`efectivo`	número	Importe aplicado que ingresa al cajón. Lo calcula una única función de reparto por bolsillo; una venta a crédito lo deja en cero.
`totalTransferenciaPago`	número	Importe aplicado que ingresa por medios electrónicos, con la misma regla única de reparto.

Campo	Tipo	Contenido
`montoRecibido`	número	Importe entregado por el cliente en efectivo, que puede superar el total y dar lugar a vuelto.
`cambio`	número	Vuelto entregado. Solo el efectivo puede dar lugar a vuelto.
`cobrado`	número	Importe ya cobrado de una venta a crédito. Una venta a crédito nace con este campo en cero, porque es el valor que alimenta el saldo pendiente del cliente. Se incrementa con cada cobro posterior.
`pagado`	booleano	Verdadero cuando lo cobrado alcanza el total. Solo aparece en los documentos que pasaron por cobros parciales.
`cliente_nombre`	texto	Nombre o razón social del receptor. Adopta el texto «-» cuando no se identifica al cliente.
`cliente_dni`	texto	Documento del receptor. Es el campo por el que se consultan los documentos de un cliente y el que alimenta el cálculo de su saldo.
`cliente_direccion`	texto	Dirección del receptor, obligatoria en la factura y opcional en el resto.
`referencia`	texto	Número de operación o referencia declarada en un pago por transferencia.
`envio`	objeto	Datos de entrega a domicilio: `tipo`, `direccion`, `referencia`, `zonaId`, `costo`, `notas` y `estado`. Se agrega con propagación condicional, de modo que su ausencia no introduce un valor indefinido.
`vendedor`	texto	Nombre de quien armó la venta. Un pedido lo escribe al crearse; al cobrarlo se conserva, para que el comprobante distinga a las dos personas que intervinieron.
`cajero`	texto	Nombre de quien cobró. En un pedido nace vacío y se completa al cobrarlo, junto con `vendedor`.
`sucursal`	texto	Nombre de la sucursal o establecimiento, tomado de la configuración.
`mensaje`	texto	Leyenda impresa al pie del comprobante.
`son_letras`	texto	Importe total expresado en letras.
`serie_numero`	texto	Número completo del comprobante dentro del bloque de venta, necesario porque el comprobante se imprime a partir de este bloque. En un pedido contiene el código del pedido y, tras el cobro, se reescribe con el número del comprobante emitido.
`turnoId`	texto	Turno en el que se registró la venta, replicado dentro del bloque.

3.4 La forma de un ítem: dos representaciones distintas

Una misma línea de venta adopta dos representaciones diferentes según el momento del flujo. La distinción no es estilística: responden a necesidades opuestas y confundirlas produce comprobantes que declaran importes que nadie cobró.

Forma de carrito

Es el estado interno de la pantalla de venta. Usa `quantity` como cantidad e interpreta `precioVenta` como el precio de **una presentación**, no de una unidad base. Conserva `presentacionNombre`, `presentacionId`, `factorConversion`, `usarPrecioEspecial` y la marca `manual` de los ítems que no pertenecen al catálogo.

Forma de documento

Es la que se persiste en `tblDocumentos`. Expresa todo en **unidades base**: `cantidad` y `cantidadEnUnidadBase` cuentan unidades base, y `precioVenta` es el precio de una unidad base. Incorpora `subtotal`, `unidad`, `categoria` y `tipoAfectacion`.

Campo de la línea	Forma de carrito	Forma de documento
`id`	Sí	Sí. Identificador del producto, o identificador propio con el prefijo `manual-` cuando el ítem no pertenece al catálogo.
`nombre`	Sí	Sí.
`quantity`	Sí: cantidad de presentaciones	Solo en los pedidos, donde se escribe igual a la cantidad en unidades base.
`cantidad`	No	Sí: cantidad en unidades base.
`cantidadEnUnidadBase`	No	Sí: cantidad en unidades base, usada al revertir existencias en una anulación.
`precioVenta`	Sí: por presentación	Sí: por unidad base.
`subtotal`	No	Sí: importe de la línea, calculado como precio unitario por cantidad.
`unidad`	No	Sí: rótulo de la unidad vendida, por defecto «Un».
`presentacionNombre`	Sí	Solo en los pedidos, donde replica el rótulo de la unidad.
`presentacionId`	Sí	Sí. Adopta el valor `unidad` cuando no hay presentación.
`factorConversion`	Sí	Sí: unidades base contenidas en la presentación vendida.
`categoria`	Sí	Sí. Los ítems manuales se agrupan en la categoría reservada para «Varios», de modo que no recaigan en «Sin categoría» en los reportes, que leen este campo como dato histórico.
`tipoAfectacion`	Sí	Sí: código de afectación al impuesto general a las ventas de la línea. Es el campo decisivo del comprobante.
`usarPrecioEspecial`	Sí	No.
`manual`	Sí	No. La condición de ítem manual se reconoce por el prefijo de su identificador.
`kitItems`	Sí	Solo en los pedidos, para que el cobro descuente las existencias de los componentes.

Por qué la distinción es obligatoria

El motivo es tributario y de inventario a la vez, y está documentado en el propio código como consecuencia de un defecto detectado y corregido:

- **La afectación al impuesto vive en la línea del documento.** El comprobante decide si una línea está gravada, exonerada o inafecta leyendo `tipoAfectacion`. Si ese campo falta, el sistema aplica el valor por defecto del negocio y, en una región declarada exonerada, una línea sin dato pasaría a declararse gravada, agregando un 18 % que nadie cobró. Por ese motivo la conversión de carrito a documento —tanto en la venta directa como en el envío de un pedido a caja— debe copiar el formato del documento, incluida la afectación de cada producto.
- **El precio debe referirse a una unidad inequívoca.** Un producto con presentaciones tiene un precio por presentación y un factor de conversión mayor que uno. Si la línea del documento conservara el precio de la presentación junto con una cantidad expresada en unidades base, el `subtotal` quedaría multiplicado por el factor: una compra de dos cajas de doce unidades declararía veinticuatro veces el importe de una caja.

> **Caso verificado en el código.** El flujo de envío de un pedido a caja llegó a persistir las líneas con la forma del carrito, sin `tipoAfectacion`. Al cobrar el pedido, el comprobante declaraba gravado un producto exonerado y sumaba 18 % de impuesto que nadie había cobrado; con una venta por presentación, el `subtotal` además resultaba inflado por el factor de conversión. La corrección consistió en que el envío a caja construya las líneas con la forma del documento, escribiendo de manera simultánea la cantidad, la cantidad en unidad base, el precio por unidad base, el subtotal, la categoría y la afectación.

Un defecto de la misma familia afecta al descuento: cuando las líneas se declaran a precio de lista, el total debe acompañarse de la fila de descuento que explique la diferencia. El caso documentado en el código es una venta con una promoción de volumen —dos unidades de un producto a precio promocional y dos unidades de otro, con líneas que suman 2,20 y un total de 2,00—: si el descuento se pierde, el comprobante muestra líneas que suman más que el total sin ninguna fila que lo justifique, y el archivo enviado a la administración tributaria declararía 2,20. Por eso la lectura del descuento de una venta está centralizada en una única función que acepta tanto el nombre en notación de camello como el nombre antiguo en notación con guion bajo.

Finalmente, debe señalarse que la forma de carrito **no se persiste en la base de datos**: vive en el estado de la pantalla de venta y, cuando el cajero pausa una venta, en el almacenamiento local del navegador. Ninguna de las dos representaciones se escribe nunca en `tblDocumentos`.

4. Ciclo de vida del documento

El ciclo de vida describe cómo un documento de `tblDocumentos` cambia de estado desde que un vendedor lo envía a caja hasta que se convierte en comprobante, se descarta o se anula. La lectura del estado es estricta: cada transición verifica el estado previo dentro de una transacción de base de datos y rechaza la operación si el documento ya no se encuentra en el estado esperado.

PENDIENTE — pedido enviado a caja

`tipo: 'PENDIENTE'` · `tipoDoc: 'PENDIENTE'` · `serie: ''` · `numero: 0` · `correlativo: 0` · `serieNumero: 'PED-000123'` · `estado: 'pendiente'` · `cajaId` (ruteo) · `vendedorId` · `vendedorNombre` · sin bloque `sunat` · sin forma de pago.

▼

pagado

El cajero cobra el pedido y elige el comprobante. Se asignan serie y correlativo una sola vez, se fija la forma de pago real, se descuenta el inventario y se registra la venta en el turno.

cancelado

Pedido descartado antes de cobrar, con motivo obligatorio. No existe comprobante que anular ni existencias que reponer.

anulada

Comprobante emitido y luego anulado. Cuando la anulación se formaliza ante la administración tributaria, se emite una nota de crédito o de débito vinculada al comprobante original.

▼

Estados terminales

Si el pedido se cobra, el documento pasa a `'pagado'` y su `tipoDoc` adopta el comprobante elegido: `'NV'` o `'BOLETA'` o `'FACTURA'`. Los pedidos descartados quedan en `'cancelado'` y los comprobantes anulados en `'anulada'`. Un documento cancelado o anulado no vuelve a ningún otro estado, y las consultas de pedidos pendientes solo consideran los documentos cuyo estado es `'pendiente'`.

4.1 Transiciones y campos afectados

Transición	Disparador	Campos que cambian	Qué no se toca
Creación del pedido	El vendedor envía la venta a caja desde el punto de venta.	Se crea el documento con `tipo` y `tipoDoc` en `'PENDIENTE'`, `serie` vacía, `numero` y `correlativo` en cero, `serieNumero` con el código `PED-…`, `estado: 'pendiente'`, `vendedorId`, `vendedorNombre` y `cajaId` de ruteo. El bloque `venta` nace sin forma de pago, con pagos vacíos y con el importe total ya neto de descuentos.	El inventario, que no se descuenta. Las series y los correlativos, que no se consumen. El turno de caja, que no registra la operación.
Pedido → `pagado`	El cajero cobra el pedido y elige comprobante y forma de pago.	`estado` pasa a `'pagado'`; `tipo`, `tipoDoc`, `serie`, `numero`, `correlativo` y `serieNumero` adoptan el comprobante emitido; `pedido` conserva el código `PED-…`; `cajero`, `cajeroId`, `fechaCobro` y `turnoId` se completan; `usuarioId` pasa a ser el del cajero y `vendedorNombre` permanece; se incorpora o actualiza el bloque `sunat` —o se anula de forma explícita cuando el comprobante es una nota de venta—; dentro de `venta` se escriben `tipoPago`, `forma_pago`, `pagos`, `montoRecibido`, `cambio`, `efectivo`, `totalTransferenciaPago`, `serie_numero` y las dos personas intervinientes.	El importe total, que es el cotizado por el vendedor y no se recalcula. La afectación y el descuento ya declarados en las líneas.
Pedido → `cancelado`	El cajero descarta el pedido, con motivo obligatorio.	`estado` pasa a `'cancelado'`; se escriben `motivoCancelacion`, `updatedAt` y el bloque `anulacion` con `fecha`, `usuario`, `motivo`, `comprobante: null` y `tocoStock: false`. La operación se registra en la bitácora de auditoría.	**El inventario**: no hay existencias que reponer, porque el pedido nunca las descontó. **Las series y los correlativos**: el pedido nunca consumió un número, de modo que descartarlo no deja huecos en la numeración. El documento tampoco se elimina: queda persistido como registro histórico.

Transición	Disparador	Campos que cambian	Qué no se toca
Comprobante → `anulada`	Anulación de una nota de venta del día o de un comprobante electrónico que todavía no fue aceptado por la administración tributaria. Un comprobante ya aceptado no se anula: se emite una nota de crédito.	`estado` pasa a `'anulada'`; se escriben `anulacion` —con `fecha`, `usuario`, `motivo` y `documentoAnuladoRef` — y `updatedAt`. Se crean movimientos de inventario de tipo `'anulacion_venta'`, que devuelven las existencias, y se ajusta el turno: si sigue abierto se descuentan los importes de sus acumulados, y si ya está cerrado se acumulan en los totales de anulación.	**El correlativo no se reutiliza**: el número emitido queda consumido y anularlo no devuelve la secuencia. El documento no se elimina.
Comprobante → nota de crédito o débito	Anulación formal de un comprobante ya aceptado, o devolución de mercadería.	Se crea un **documento nuevo** con `tipo` y `tipoDoc` en `'NC'` o `'ND'`, su propio `serie`, `numero`, `correlativo` y `serieNumero`, y un bloque `venta` que referencia el comprobante original mediante los campos de documento de referencia y de tipo de documento original. La nota de crédito resta del total del día y devuelve el importe en la misma proporción de bolsillos en que se cobró el comprobante original; la nota de débito no mueve la caja.	El documento original no se modifica. Su correlativo tampoco se libera.
Nota de venta → boleta consolidada	Consolidación de las notas de venta del día para declararlas ante la administración tributaria.	Se crea una boleta con `consolidado: true` y `nvsConsolidadas` con los identificadores de las notas agrupadas; cada nota recibe en su bloque `sunat` las marcas `consolidado`, `consolidadoAt` y `documentoConsolidadoId`.	El estado de las notas de venta, que no cambia, y su numeración, que queda consumida. Una nota ya consolidada no admite modificación ni anulación.

Efecto de los estados sobre los reportes. Los reportes fiscales filtran los documentos por su tipo contra el conjunto de comprobantes válidos —nota de venta, boleta, factura, nota de crédito y nota de débito—, de modo que los pedidos y los documentos cancelados quedan excluidos por construcción, sin necesidad de una regla adicional. Los documentos anulados se excluyen por su estado, y las notas de crédito restan del total mientras las notas de débito suman.

5. Contadores de correlativos (`tblContadores`)

La colección `tblContadores` no guarda documentos de negocio sino secuencias de numeración. Su estructura es mínima y deliberadamente uniforme: un documento por espacio de numeración, con el campo `ultimoCorrelativo` como único valor relevante.

5.1 Dos espacios de numeración

La numeración de comprobantes se reparte en dos espacios independientes, porque una nota de venta y un comprobante electrónico son documentos de naturaleza distinta y no pueden compartir secuencia aunque su serie se llame igual:

- **Espacio de notas de venta**, con la clave `notaVenta_{serie}`. Agrupa las notas de venta, que no se declaran ante la administración tributaria.
- **Espacio de comprobantes electrónicos**, con la clave `sunat_{serie}`. Agrupa las boletas, las facturas y las notas de crédito y débito.

La clave nunca se compone a mano. El identificador se obtiene siempre de la función `claveContador(tipoDoc, serie)`, declarada en `src/lib/contadores.ts`, que resuelve el espacio a partir del tipo de documento —solo la nota de venta queda fuera del espacio de comprobantes electrónicos— y concatena la serie ya resuelta. El nombre del campo también está declarado como constante en el mismo módulo, de modo que ningún punto del sistema escriba un contador con un nombre alternativo.

Identificador del documento	Espacio	Numeración que gobierna
`notaVenta_NV01`	Notas de venta	Notas de venta de la serie `NV01`.
`sunat_B001`	Comprobantes electrónicos	Boletas de la serie `B001`, incluidas las boletas consolidadas emitidas con esa serie.
`sunat_F001`	Comprobantes electrónicos	Facturas de la serie `F001`.
`sunat_BC01`	Comprobantes electrónicos	Notas de crédito de boletas.
`sunat_BD01`	Comprobantes electrónicos	Notas de débito de boletas.
`sunat_FC01`	Comprobantes electrónicos	Notas de crédito de facturas.
`sunat_FD01`	Comprobantes electrónicos	Notas de débito de facturas.
`pedido`	Pedidos	Códigos de pedido `PED-000123`, en un espacio propio y separado de toda serie fiscal.

Cada identificador de la tabla anterior es el resultado de aplicar la función de composición de clave al tipo de documento y a la serie correspondiente: no son cadenas literales escritas en el código. Las series que aparecen son los valores por defecto declarados en el sistema; en operación

provienen de la caja del usuario que emite o, en su defecto, de la configuración de la empresa. Los ejemplos de notas de crédito y débito corresponden a los valores por defecto de esas cuatro series.

5.2 El campo `ultimoCorrelativo` y la asignación atómica

El campo `ultimoCorrelativo` contiene el último número emitido en ese espacio. La asignación del siguiente número se realiza siempre dentro de una transacción de base de datos, junto con el documento que lo consume, de modo que no puede ocurrir que un contador avance sin que exista el comprobante que lo justifica, ni que dos emisiones simultáneas obtengan el mismo número. El contador se escribe con `ultimoCorrelativo` y una marca de actualización; en los flujos de pedidos y de notas se escribe además el campo `serie`, que documenta a qué serie corresponde el documento contador.

Si el documento contador no existe —por una restauración de la base, por un contador eliminado o porque la serie fue numerada por otra aplicación—, el sistema no reinicia la secuencia en uno. Antes de la transacción realiza una **lectura de solo lectura** sobre los documentos ya emitidos para determinar el máximo correlativo existente de esa serie y con ese tipo, y siembra el contador desde ese máximo dentro de la misma transacción que crea el comprobante. El objetivo es evitar que una nota vuelva a numerarse como la primera de la serie y sea rechazada por duplicada.

5.3 El contador aparte de pedidos

Los pedidos no consumen correlativos fiscales. Su código se obtiene de un contador propio, cuya clave es la constante `pedido`, y se formatea con la función `codigoPedido(numero)`, que produce una cadena con el prefijo `PED-` y el número rellenado con ceros a la izquierda hasta seis dígitos: `PED-000123`.

Esta separación es la que garantiza que un pedido descartado no deje huecos en las series fiscales. El número de comprobante se asigna **una sola vez, al cobrar**, a partir del contador del espacio que corresponde al tipo elegido por el cajero; mientras el pedido espera en caja, solo tiene un código de pedido y ningún número fiscal.

> **Riesgo documentado en el código.** El módulo de contadores consigna un defecto real del flujo de pedidos: al utilizar para todas las clases de documento la clave del espacio de comprobantes electrónicos y al escribir el campo del contador con un nombre distinto del declarado, se creó un contador paralelo que arrancaba en uno mientras las notas de venta vivas seguían numeradas en un rango muy superior. La consecuencia habría sido que la siguiente nota de venta pendiente repitiera un número ya emitido. La regla que se desprende —usar siempre la función de composición de clave y la constante del campo, y nunca escribir ninguno de los dos a mano— está verificada en el código actual como un único punto de definición.

6. Índices compuestos

6.1 Por qué se necesitan

Cloud Firestore resuelve automáticamente las consultas que filtran o ordenan por un único campo, y también las combinaciones de varios filtros de igualdad. En cambio, una consulta que combine un filtro de igualdad con un rango o un ordenamiento sobre otro campo exige un **índice compuesto** declarado de antemano. Si el índice no existe, la consulta no devuelve resultados parciales ni una lista vacía: falla con el mensaje *«The query requires an index»*. Se trata, por lo tanto, de un requisito de despliegue y no de una optimización opcional.

El sistema declara sus índices en el archivo `firestore.indexes.json`, que la configuración de despliegue de Firebase (`firebase.json`) asocia al proyecto. Todos los índices declarados son de ámbito de colección. En la Tabla 6.1 se transcriben los quince índices declarados, con el patrón de consulta que cada uno habilita y el punto del código donde se verificó.

Colección	Campos y orden	Consulta que lo requiere
`tblDocumentos`	`usuarioId` asc · `fecha` desc	Listado de documentos de un usuario en un rango de fechas, ordenado del más reciente al más antiguo. El módulo de reportes lo consulta cuando el usuario tiene restringida la vista a sus propias ventas.
`tblDocumentos`	`tipoDoc` asc · `anioMes` asc · `fecha` desc	Documentos de un tipo dentro de un período, del más reciente al más antiguo: es la consulta del listado de comprobantes del mes.
`tblDocumentos`	`tipoDoc` asc · `anioMes` asc · `fecha` asc	La misma consulta en orden cronológico ascendente, que es la que requieren los reportes tributarios y el reporte por empleado, donde el orden de presentación es el de emisión.
`tblDocumentos`	`tipoDoc` asc · `anioMes` asc · `usuarioId` asc · `fecha` asc	Documentos de un tipo, de un período y de un usuario, en orden cronológico. Es la variante restringida de la consulta anterior para el usuario sin vista global.
`tblDocumentos`	`tipoDoc` asc · `fecha` desc	Documentos de un tipo a partir de una fecha, del más reciente al más antiguo. Se utiliza en la consulta de las últimas ventas y en la rotación de inventario.
`tblDocumentos`	`tipoDoc` asc · `fecha` asc	La misma consulta en orden cronológico ascendente.
`tblDocumentos`	`tipoDoc` asc · `usuarioId` asc · `fecha` desc	Documentos de un tipo y de un usuario desde una fecha determinada, del más reciente al más antiguo.
`tblDocumentos`	`venta.cliente_dni` asc · `usuarioId` asc	Documentos de un cliente, y su variante restringida a un usuario. Sobre esta consulta se construyen el historial del cliente y el cálculo de su saldo de crédito.
`tblDocumentos`	`cajaId` asc · `estado` asc · `fecha` desc	Documentos de una caja filtrados por estado, del más reciente al más antiguo. En la revisión del código correspondiente a este anexo la lista de pedidos se resuelve con una igualdad simple sobre el estado y el reparto por caja se decidió en memoria; el índice se conserva declarado por corresponder al despliegue vigente.
`tblTurnos`	`estado` asc · `fechaCierre` asc	Turnos cerrados dentro de un rango de fechas de cierre, para el consolidado de caja del día y el panel de operaciones.
`tblTurnos`	`usuarioId` asc · `fechaApertura` asc	Turnos de un usuario dentro de un rango de fechas de apertura, para el arqueo del día restringido a un cajero.
`tblMovimientos`	`tipoMovimiento` asc · `fecha` asc	Movimientos de un tipo dentro de un rango de fechas: compras del período y su análisis por proveedor.

Colección	Campos y orden	Consulta que lo requiere
`tblMovimientos`	`productoId` asc · `fecha` asc	Kardex de un producto dentro de un rango de fechas, ordenado cronológicamente para reconstruir el saldo corriente.
`tblEgresos`	`proveedorId` asc · `categoria` asc	Pagos registrados a un proveedor, identificados por su categoría.
`tblHistorialPrecios`	`productoId` asc · `fecha` desc	Historial de precios de un producto, del cambio más reciente al más antiguo.

6.2 Despliegue de los índices

Los índices se despliegan con el comando `npx firebase deploy --only firestore:indexes`, que publica exactamente el contenido del archivo de declaración. Al agregar una consulta que combine igualdad con rango u ordenamiento, la práctica establecida en el proyecto es declarar el índice en el archivo y desplegarlo en el mismo cambio; de lo contrario la consulta falla en producción con el mensaje de índice faltante.

> **Advertencia sobre la opción de forzado.** No debe utilizarse el despliegue forzado de índices. La operación con forzado elimina los índices del proyecto que no estén declarados en el archivo de declaración, y el proyecto mantiene un índice creado de forma manual cuyo borrado provocaría la caída de la consulta que lo utiliza. La instrucción vigente es desplegar exclusivamente con el alcance de índices y sin la opción de forzado.

6.3 Ordenamiento en el cliente como alternativa

Existe un comportamiento del motor que condiciona el diseño de varias consultas y que conviene consignar: un ordenamiento por un campo **descarta los documentos que no contienen ese campo**. Un documento antiguo, o escrito por un flujo que no lo completaba, desaparecería silenciosamente del listado en lugar de aparecer al final. Por ese motivo, cuando el campo de orden puede faltar, la consulta se resuelve sin ordenamiento y la lista se ordena en memoria. Este criterio está aplicado, de manera verificada, en el listado de categorías —cuyo campo de orden es opcional—, en la lista de pedidos pendientes y en el ordenamiento de turnos del arqueo diario.

7. Reglas de integridad que impone la aplicación

Cloud Firestore no aplica restricciones de esquema, de tipo ni de integridad referencial. En un modelo documental, las garantías que en un motor relacional aportarían las restricciones declarativas deben ser impuestas por la aplicación. Las reglas que se enumeran a continuación son las que el sistema verifica efectivamente en su código y se presentan con la justificación de cada una; varias de ellas surgieron de defectos reales detectados en operación, que se consignan porque explican el motivo de la regla.

1. **Las escrituras relacionadas se agrupan en una transacción o en un lote.** Registrar una venta no es una escritura sino varias: el correlativo, las existencias de cada producto, los movimientos de kardex, el documento y los acumulados del turno. Todas se ejecutan dentro de una transacción de base de datos, de modo que el resultado sea «o todo, o nada». El motivo es doble: una interrupción a mitad del proceso dejaría un correlativo consumido sin comprobante, o existencias descontadas sin la venta que las explique. Como contrapartida, el motor exige que *todas las lecturas se realicen antes de la primera escritura*; el código respeta esa disciplina de forma explícita, separando cada transacción en una fase de lecturas y una fase de escrituras. Los procesos masivos —el renombrado de una categoría, la transferencia de pedidos entre cajas y la limpieza del registro de errores— se ejecutan por lotes cuyo tamaño no supera el límite de quinientas operaciones que admite el motor, y devuelven la cantidad realmente afectada, para no informar un éxito parcial como si fuera total.

2. **Ningún campo puede escribirse con un valor indefinido.** Firestore no ignora el valor indefinido: rechaza la escritura completa con un error de valor de campo no admitido. Y el compilador de TypeScript no advierte el problema, porque un campo opcional declarado como «texto o indefinido» satisface la verificación de tipos y falla recién al ejecutarse, en producción y frente al usuario. El sistema aplica tres capas de defensa: los campos opcionales se agregan con propagación condicional o se normalizan a nulo; en el borde de escritura, todo objeto que se persiste se envuelve en una función de limpieza que elimina recursivamente las propiedades indefinidas, descarta los valores indefinidos sueltos dentro de listas y respeta los valores nulos, las fechas y los centinelas del motor; y existe además una función de diagnóstico que devuelve la ruta del primer campo indefinido de un objeto, para localizar el origen antes de intentar la escritura.

   > **Caso real documentado.** El flujo de envío de una venta a caja **fallaba siempre**: el objeto enviado incluía dos campos con valor indefinido y el motor rechazaba la transacción completa con el mensaje relativo a un valor de campo no admitido. La verificación de tipos estaba en verde y el defecto no se advertía en desarrollo. La consecuencia operativa fue que el flujo nunca había funcionado y no existía un solo pedido registrado. La corrección combinó las tres capas descritas. Un defecto análogo se produjo en el guardado del estado de un comprobante ante la administración tributaria, donde la respuesta del servicio no incluye siempre todos los campos: el fallo al guardar el estado ocultaba el resultado real del envío, que ya había sido aceptado. La conclusión que el código documenta es que la limpieza evita que la operación *falle*, pero no sustituye la corrección del origen: un campo que no debería faltar sigue siendo un defecto de datos.

3. **Cuando el campo de ordenamiento puede faltar, el orden se resuelve en el cliente.** Un ordenamiento por un campo descarta los documentos que no lo contienen, tal como se explicó en el apartado 6.3. La regla es no ordenar en la consulta cuando el campo es opcional, y ordenar la lista en memoria una vez recuperada, aun cuando ello implique traer la colección

completa. Se aplica a colecciones pequeñas y de escritura poco frecuente, donde el costo es despreciable frente al riesgo de perder documentos del listado de forma silenciosa.

4. **El arqueo se guarda como registro histórico y no se reescribe.** Al cerrar un turno, la transacción escribe un documento nuevo en `tblCierresCaja` con la fotografía completa del turno —apertura, cierre, acumulados, el detalle de las ventas y de los gastos, el efectivo esperado, el efectivo contado, la diferencia y el análisis de las causas— y actualiza el turno únicamente con el estado de cierre y sus totales de cuadre. El cierre, por lo tanto, no se recalcula después: queda como registro de lo que efectivamente ocurrió, y su resultado es auditable. La corrección de un turno histórico sigue una regla restrictiva: se modifica únicamente el campo necesario, se deja constancia escrita en el campo de observaciones y se registra la operación en la bitácora de auditoría.

7.1 Reglas de negocio sostenidas por el modelo

A las reglas anteriores se suman cuatro decisiones de diseño que el propio modelo de datos materializa y que conviene consignar porque explican por qué los campos están dispuestos como están:

1. **El precio del producto es un precio final al público**, con el impuesto general a las ventas incluido. El modelo no almacena un precio sin impuesto que se recargue al cobrar. La desagregación del impuesto se realiza al construir el comprobante y solo en las líneas cuya afectación es gravada.
2. **El comprobante declara lo que se cobra.** El descuento se reparte entre las líneas antes de despejar el impuesto, y las líneas conservan su precio de lista. El modelo tributario peruano contempla el descuento como una fila propia del comprobante y no como un precio unitario ya rebajado; por eso el importe descontado se declara de forma explícita y las líneas no se reescriben a un precio distinto del de lista.
3. **La afectación al impuesto es un atributo de cada producto** y prevalece sobre cualquier parámetro general del negocio. La marca de región exonerada de la configuración no actúa como una anulación de la afectación declarada: es únicamente el valor por defecto de las líneas que no traen dato propio. La consecuencia práctica es que en una región exonerada un comprobante no debe mostrar líneas de operaciones gravadas, y esa comprobación se realiza sobre el documento impreso.
4. **El crédito y la cancelación de un pedido tienen consecuencias distintas sobre el inventario.** Una venta a crédito no ingresa dinero a ningún bolsillo de caja, porque queda pendiente de cobro; su cobro posterior sí lo hace, en el bolsillo del medio con que se cobró. Un pedido descartado no repone existencias, porque nunca las descontó. Ambas reglas se apoyan en la misma premisa: cada efecto sobre la caja y sobre el inventario tiene un único lugar en el código que lo produce.

8. Volumen y operación real

El sistema se encuentra en operación efectiva en un establecimiento comercial, con uso diario del punto de venta, del control de existencias, de la gestión de caja y de la emisión de comprobantes electrónicos. Esa condición de operación es la que da sentido a las reglas de integridad de la sección anterior: la mayor parte de ellas no proviene de un ejercicio de diseño teórico, sino de defectos observados sobre datos reales y corregidos en el propio modelo, tal como se documenta en los apartados correspondientes.

8.1 Tamaño del modelo documentado

16	2	311	15
COLECCIONES DECLARADAS	COLECCIONES AUXILIARES	CAMPOS DOCUMENTADOS	ÍNDICES COMPUESTOS

El recuento de campos se compone de 236 campos correspondientes a las quince colecciones distintas de `tblDocumentos` —incluidos los tres campos del contador—, 27 campos de cabecera de `tblDocumentos`, 32 campos del bloque `venta` y 16 campos de la forma del ítem del documento. El total asciende a 311 campos distintos. Cada cifra coincide con el recuento de las tablas de este anexo: la Tabla 2.2 suma 236 campos en sus quince filas y la sección 3 documenta los 75 restantes de la colección de documentos.

8.2 Sobre las métricas de producción

Este anexo no incluye cifras de producción —cantidad de comprobantes emitidos, de turnos cerrados, de productos en catálogo ni de movimientos registrados—. La razón es metodológica: esas cifras no son deducibles de la lectura del código fuente, y el proyecto tiene establecido que la consulta de datos de producción se realiza únicamente en modo de solo lectura, de forma acotada y con propósitos de diagnóstico, sin dejar credenciales activas ni datos extraídos. Consignar una cifra de esta clase sin haberla obtenido por ese procedimiento equivaldría a presentar una estimación como una medición.

Lo que sí puede afirmarse con respaldo en el código es la **forma** que tienen los datos en operación, y las secciones anteriores la documentan: la colección de contadores demuestra que la numeración de notas de venta y la de comprobantes electrónicos avanzan por espacios separados; la existencia de una función de limpieza de valores indefinidos en el borde de escritura, con su caso documentado, demuestra que el flujo de pedidos fue corregido y quedó operativo; la presencia de la conciliación por billetera en el cierre de turno demuestra que la operación distingue efectivo de medios electrónicos; y el registro de turnos cerrados con su análisis de diferencias demuestra que el control de caja es una rutina efectiva y no una previsión del diseño.

8.3 Advertencia sobre la lectura de este anexo

Un modelo documental no se valida únicamente por la coherencia de sus nombres, sino por la disciplina con que la aplicación los escribe. La tercera capa de este anexo —la sección 7— es, por ese motivo, tan relevante como el inventario de colecciones: enumera las restricciones que el motor no impone y que la aplicación debe imponer por sí misma. La conclusión que se desprende del análisis es que, en ausencia de un esquema validado por el motor, la integridad del modelo depende de dos prácticas sostenidas en el tiempo: la existencia de un único punto de definición para cada

nombre y para cada cálculo, y la verificación de los efectos de cada escritura sobre las colecciones relacionadas.

9. Trazabilidad de las fuentes

Todos los nombres de colección, campos, valores enumerados, índices y comportamientos descritos en este anexo provienen de la lectura directa de los siguientes archivos del repositorio del sistema. Se consignan para permitir la verificación de cada afirmación del documento.

Archivo	Información que aporta a este anexo
`src/constants/colecciones.ts`	Registro central de los nombres de las dieciséis colecciones y nota sobre la compatibilidad con la aplicación móvil.
`src/services/ventas.ts`	Alta de venta, anulación, creación y cobro de pedidos y cancelación de pedidos: cabecera del documento, bloque de venta, forma del ítem, movimientos de existencias y ajuste del turno.
`src/services/notas.ts`	Emisión de notas de crédito y débito, contador del espacio de comprobantes electrónicos, campos de referencia al documento original y efecto en caja.
`src/services/consolidado.ts`	Boleta consolidada, campos de consolidación e identificadores de las notas agrupadas.
`src/services/clientes.ts`	Ficha de cliente, campos de crédito, cálculo del saldo y registro de cobros.
`src/services/turnos.ts`	Turnos de caja, acumulados, cierre con el documento histórico de cierre y transiciones de estado.
`src/services/egresos.ts`	Egresos, categorías, tipos de pago y registro dentro del turno.
`src/services/compras.ts`	Movimientos de entrada por compra y acumulados de gastos del turno.
`src/services/kardex.ts`	Clasificación de movimientos de entrada y salida y formato del kardex.
`src/services/cajas.ts`	Campos de la caja y sus siete series de comprobante.
`src/services/usuarios.ts`	Campos del usuario, series heredadas del contrato de series y validación de conflictos entre cajas.
`src/services/categorias.ts`	Campos de la categoría, siembra inicial, renombrado en lote y criterio de ordenamiento en memoria.
`src/services/errores.ts`	Campos del registro de errores y borrado por lotes.
`src/services/historial-precios.ts`	Campos de la bitácora de precios y su condición de escritura.
`src/services/promociones-volumen.ts`	Campos y tipos de las promociones por volumen.
`src/services/proveedores.ts`	Campos del proveedor, contactos y bloque de crédito declarado.
`src/services/sunat.ts`	Estructura y valores del bloque de estado tributario y uso de la limpieza de valores indefinidos en el borde de escritura.
`src/services/reportes.ts`	Consultas del arqueo y de los reportes, filtros por tipo de documento y ordenamiento en memoria.
`src/lib/contadores.ts`	Espacios de numeración, función de composición de clave, campo del contador y formato del código de pedido.
`src/lib/limpio.ts`	Función de limpieza de valores indefinidos, su alcance y su justificación.

Archivo	Información que aporta a este anexo
`src/lib/sunat-importes.ts`	Reparto del descuento por línea, resolución de la afectación y lectura unificada del descuento de una venta.
`src/lib/pagos.ts`	Modelo de bolsillo y medio, reparto por bolsillo y composición del rótulo de la forma de pago.
`src/lib/arqueo.ts`	Fórmula única del efectivo esperado, desglose por origen y conciliación por billetera.
`src/lib/pedidos.ts`	Reparto de pedidos por caja y adaptación de un pedido al carrito del punto de venta.
`src/lib/series.ts`	Campos de serie de usuario y de empresa y espacio de numeración de cada uno.
`src/lib/audit.ts`	Colección de bitácora de auditoría y sus campos.
`src/lib/ventas-pausadas.ts`	Confirmación de que la venta pausada se almacena en el navegador y no en la base de datos.
`src/lib/customer-display.ts`	Colección auxiliar de sesiones de pantalla al cliente.
`src/constants/tributario.ts`	Catálogo de códigos de afectación, tipos de nota y valores por defecto de la configuración del negocio.
`src/hooks/use-productos.ts`	Tipo del producto y forma de las presentaciones y de los componentes de un conjunto.
`src/hooks/use-configuracion.ts`	Documento único de configuración y escritura de parámetros.
`src/pages/Inventario.tsx`	Alta y edición de productos, ajustes de existencias y toma de inventario con sus movimientos.
`src/pages/Pdv.tsx`	Construcción de las líneas en forma de documento al enviar un pedido a caja y armado del cobro.
`src/pages/Reportes.tsx`	Consultas de documentos por tipo y período y su variante restringida por usuario.
`src/pages/ReporteSire.tsx`	Consulta de documentos por tipo y período en orden cronológico ascendente.
`src/pages/VentasPorEmpleado.tsx`	Consulta de documentos por empleado con el mismo patrón de ordenamiento.
`src/pages/RotacionInventario.tsx`	Consulta de documentos por tipo a partir de una fecha.
`src/pages/Compras.tsx`, `src/pages/DashboardOperaciones.tsx`, `src/pages/ProveedorDetalle.tsx`	Consultas de movimientos por tipo y fecha y de turnos cerrados por fecha de cierre.
`src/components/reportes/`	Consultas de movimientos de compra y lectura del bloque de crédito del proveedor.
`firestore.indexes.json`	Declaración de los quince índices compuestos.
`firebase.json`	Asociación del archivo de índices al proyecto de Firebase.

Archivo	Información que aporta a este anexo
`AGENTS.md`	Invariantes del proyecto, criterios de acceso a datos de producción y advertencia sobre el despliegue forzado de índices.

Alcance de la verificación. Los recuentos de colecciones, campos e índices fueron contrastados contra las tablas de este mismo documento: la Tabla 2.2 suma 236 campos en quince filas, la sección 3 documenta 75 campos de la colección de documentos —27 de cabecera, 32 del bloque de venta y 16 de la forma del ítem—, y ambos totales componen los 311 campos del apartado 8.1. La Tabla 6.1 enumera quince índices, coincidentes con los quince declarados en el archivo de índices. No se incluye ninguna afirmación sobre el estado de la base de datos en producción que no sea deducible del código fuente.